3. Graphs

3.1 Basic definitions

Definition 3.1 A **graph** is a pair $G = (V, E)$, consisting of finite sets V of **vertices** (often also called **nodes**) and E of **edges**. The edges indicate connections between two vertices and can be directed or undirected:

- For **directed graphs**, also called **digraphs**, there holds

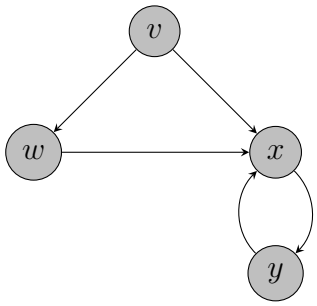
$$E \subset \{(v, w) \in V \times V : v \neq w\}.$$

  If $e = (v, w) \in E$, then v is the **initial vertex** and w the **terminal** or **end vertex** of the edge e.
- For **undirected graphs**, E is a set of disordered pairs (v, w) with $v, w \in V$ and $v \neq w$.

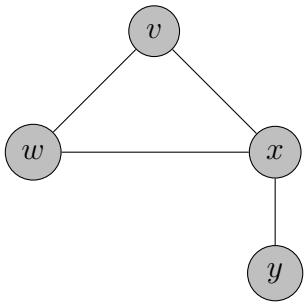
Remark It holds $(v, w) = (w, v)$ if (v, w) is an edge of an undirected graph. This is, however, not true in case of directed graphs.

Example 3.2

- Directed graph $G = \{(v, w), (w, x), (v, x), (x, y), (y, x)\}$:



- Undirected graph $G = \{(v, w), (w, x), (v, x), (x, y)\}$:



△

Remark Sometimes edges of the form (v, v) are permitted in directed graphs. They are called *loops*.

Definition 3.3 Let $G = (V, E)$ be a graph. A **path** in G is a sequence of vertices

$$\pi = v_0, v_1, \ldots, v_r$$

with $r \geq 1$ and $(v_i, v_{i+1}) \in E$, $i = 0, 1, \ldots, r-1$. We speak of a path, which goes **from v to w**, if the initial vertex is $v_0 = v$ and the endpoint is $v_r = w$. The path is **simple**, if it holds:

- either $v_0, v_1, \ldots, v_r$ are pairwise distinct
- or $v_0 = v_r$ and $v_0, v_1, \ldots, v_{r-1}$ are pairwise distinct.

The **length** $|\pi|$ of π is r, i.e., the number of edges which are contained by π. A vertex w is **reachable** from v, if there exists a path $\pi = v_0, v_1, \ldots, v_r$ in G such that $v = v_0$ and $w = v_r$.

Example 3.4 Possible paths in both graphs of Example 3.2 are

- $\pi_1 = v, w$ (length 1),
- $\pi_2 = v, w, x$ (length 2),
- $\pi_3 = v, w, x, y, x, y$ (length 5).

In the undirected case, also $\pi_4 = v, w, x, v$ (length 3) is a path. △

Definition 3.5 Let $G = (V, E)$ be a graph and $v \in V$. We define

- the set of **(direct) successors** of v by

$$\mathrm{post}(v) = \{w \in V : (v, w) \in E\},$$

- the set of **(direct) predecessors** of v by

$$\mathrm{pre}(v) = \{w \in V : (w, v) \in E\},$$

- the set of all from v reachable vertices

$$\mathrm{post}^\star(v) = \{w \in V : \text{there exists a path from } v \text{ to } w\},$$

- the set of vertices, from which v is reachable, by

$$\mathrm{pre}^\star(v) = \{w \in V : v \in \mathrm{post}^\star(w)\}.$$

A vertex $w \in \mathrm{post}(v) \cup \mathrm{pre}(v)$ is called a **neighbour** of v. The number of neighbours is the **degree** of v, where in case of directed graphs we differ between **indegree** and **outdegree**:

$$\deg(v) = \mathrm{indeg}(v) + \mathrm{outdeg}(v), \quad \mathrm{indeg}(v) = |\,\mathrm{pre}(v)|, \quad \mathrm{outdeg}(v) = |\,\mathrm{post}(v)|.$$

These definitions can be extended to sets of vertices. For example, for $W \subset V$, we have

$$\operatorname{pre}(W) = \bigcup_{w \in W} \operatorname{pre}(w).$$

Example 3.6 It holds for the directed graph from Example 3.2

$$\operatorname{post}(v) = \{w, x\}, \qquad \operatorname{post}^\star(v) = \{w, x, y\}, \qquad \operatorname{pre}(v) = \operatorname{pre}^\star(v) = \emptyset$$

and

$$\operatorname{post}(y) = \{x\}, \qquad \operatorname{post}^\star(y) = \{x, y\}.$$

Whereas, in the undirected case, we find

$$\operatorname{post}(x) = \{v, w, y\}, \qquad \operatorname{post}^\star(x) = \{v, w, x, y\}.$$

△

For undirected graphs, we have $\operatorname{post}(v) = \operatorname{pre}(v)$ and $\operatorname{post}^\star(v) = \operatorname{pre}^\star(v)$. In particular, we conclude

$$\sum_{v \in V} |\operatorname{post}(v)| = \sum_{v \in V} |\operatorname{pre}(v)| = 2|E|.$$

In the directed case, $\operatorname{post}(v)$ and $\operatorname{pre}(v)$ are in general different, but there holds

$$\sum_{v \in V} |\operatorname{post}(v)| = \sum_{v \in V} |\operatorname{pre}(v)| = |E|.$$

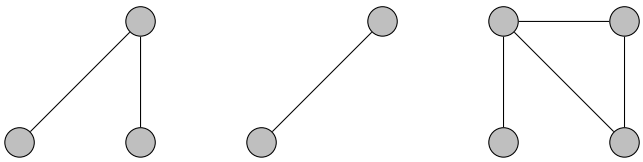
3.2 Connectivity

Definition 3.7 Let $G = (V, E)$ be an undirected graph and $C \subset V$. C is called **connected**, if there exists a patch between each pair of vertices $v, w \in C$, $v \neq w$, i.e., if $w \in \operatorname{post}^\star(v)$ and $v \in \operatorname{post}^\star(w)$. If the graph G is directed, then C is called connected, if C is connected in the underlying undirected graph.

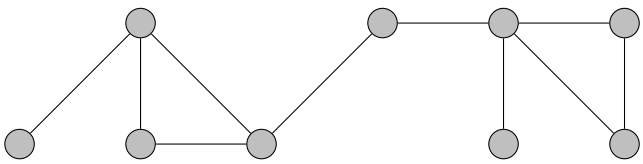
C is called a **connected component** of G if C is a nonempty connected vertex set of maximum size. Maximum size means here that C does not lie in any other connected set $C \subset C' \subset V$ and $C \neq C'$. The graph G is connected if V is connected.

Example 3.8

- A disconnected graph with three connected components is:



- A connected graph is:



△

Obviously, the connected components of an undirected graph define so-called *equivalence classes* of the vertex set V under the reachable relation "$\equiv$", where

$$v \equiv w \iff v \cup \operatorname{post}^\star(v) = w \cup \operatorname{post}^\star(w).$$

This means that any graph G can be decomposed into pairwise disjunct connected components $C_1, C_2, \ldots, C_r$ such that

$$V = \bigcup_{i=1}^{r} C_i, \qquad E = \bigcup_{i=1}^{s} E_i,$$

where $E_i := E \cap (C_i \times C_i)$.

Theorem 3.9 Let $G = (V, E)$ be an undirected graph with $n = |V| \geq 1$ vertices and $m = |E|$ edges. If G is connected, then there holds $m \geq n - 1$.

Proof. We proof the assertion by induction over n. When $n = 1$, then it follows $m = 0 = n - 1$; when $n = 2$, then G is connected if and only if $m = 1 = n - 1$. We next assume that $n \geq 3$ and that G is connected. Choose $v \in V$ such that

$$|\operatorname{post}(v)| = \min_{w \in V} |\operatorname{post}(w)| =: k.$$

Then, there holds $k > 0$ since otherwise v would be an isolated vertex, which is in contradiction to G connected. In case of $k \geq 2$, there holds

$$2m = 2|E| = \sum_{w \in V} \underbrace{|\operatorname{post}(w)|}_{\geq k} \geq n \cdot k \geq 2 \cdot n$$

and hence $m \geq n \geq n - 1$.
For $k = 1$, we infer the assertion as follows: Let $G' = (E', V')$ denote the graph which is generated by removing the vertex v and the respective outgoing edge. With G also G' is connected and, according to induction hypothesis, it holds

$$m - 1 = |E'| \geq |V'| - 1 = (n - 1) - 1 = n - 2.$$

This implies $m \geq n - 1$. □

3.3 Cyclic graphs

Definition 3.10 A **cycle** or **simple circuit** of a graph $G = (V, E)$ is a simple path $\pi = v_0, v_1, \ldots, v_r$ with $v_0 = v_r$ and $r \geq 2$ (directed graph) and $r \geq 3$ (undirected graph), respectively. A **circuit** is a path $v_0, v_1, \ldots, v_r$, which consists of several cycles.

Remark This definition means formally:

(i) each cycle is a circuit,

(ii) if $\pi_1 = v_0, v_1, \ldots, v_r$ and $\pi_2 = w_0, w_1, \ldots, w_\ell$ are circuits with $v_i = w_0 = w_\ell$, then also

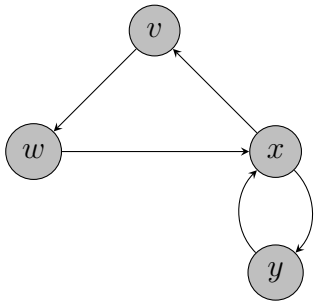
$$\pi = v_0, v_1, \ldots, v_{i-1}, w_0, w_1, \ldots, w_\ell, v_{i+1}, v_{i+2}, \ldots, v_r$$

is a circuit,

(iii) only the paths generated by (i) and (ii) are circuits.

Example 3.11

- The directed graph



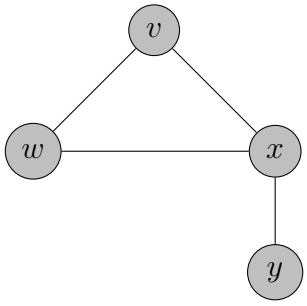
has the cycles

$$\pi_1 = x, y, x, \qquad \pi_2 = v, w, x, v,$$

and the (non-simple) circuits

$$\pi_3 = x, y, x, y, x, \qquad \pi_4 = v, w, x, y, x, v.$$

- The undirected graph



has the cyles $\pi_1 = v, w, x, v$. Note, however, that the paths $\pi_2 = x, y, x$ and $\pi_3 = v, w, x, y, x, v$ define no circuits!

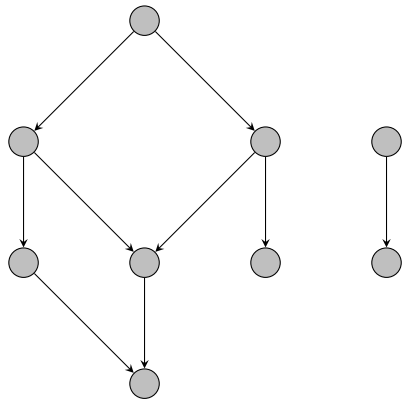
△

3.4 Trees

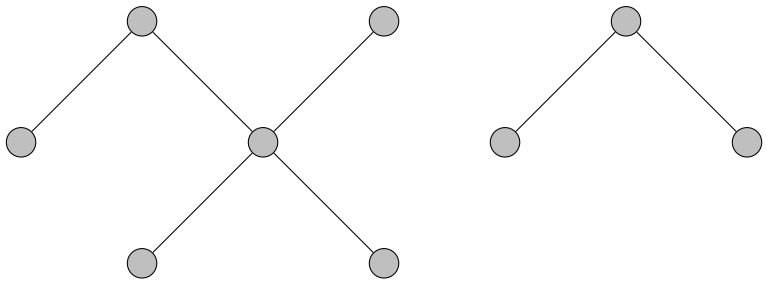
Definition 3.12 A graph G is called **acyclic** if it has no cycles. An undirected, acyclic, and connected graph is a **tree**.

Example 3.13

- Acyclic, directed graph:



- Acyclic, undirected graph:



△

Theorem 3.14 Let $G = (V, E)$ be an undirected graph with n vertices. Then, the following propositions are equivalent:

1. G is a tree.
2. G has $n - 1$ edges and is connected.
3. G hat $n - 1$ edges and is acyclic.
4. G is acyclic and insertion of an arbitrary edge generates a cycle.
5. G is connected an removing an edge results in a disconnected graph.
6. Each pair of distinct vertices of G is connected by exactly one simple path.

Proof.

$1 \Rightarrow 6$**:** This follows from the fact that the union of two different simple paths with identical initial vertices and end vertices defines a cycle.

$6 \Rightarrow 5$**:** G is connected per definition. If the edge (v, w) is removed, then w is not reachable from v.

$5 \Rightarrow 4$**:** G is acyclic, since otherwise an edge can be removed such that G is still connected. As we always find a path in G from v to w, the insertion of the edge (v, w) generates a cycle.

$4 \Rightarrow 3 \Rightarrow 2$**:** The assertion follows if

$$n = m + p \tag{3.1}$$

holds for an acyclic, undirected graph, where $m = |E|$ and p is the number of connected components. Since (3.1) is obvious for $m = 0$, we assume that (3.1) holds for $|E| = m$. Insertion of an additional edge, i.e., $|E| = m + 1$, reduces p by one as otherwise a cycle would be generated.

$2 \Rightarrow 1$**:** We destroy circuits of G by removing edges. After removing k edges, it follows from (3.1) that

$$\underbrace{n-1-k}_{\text{edges}} + \underbrace{p}_{=1} = n,$$

which means $k = 0$.

□

3.5 Implementation of graphs

The easiest way to store graphs in a computer is by using the *adjacency matrix*.

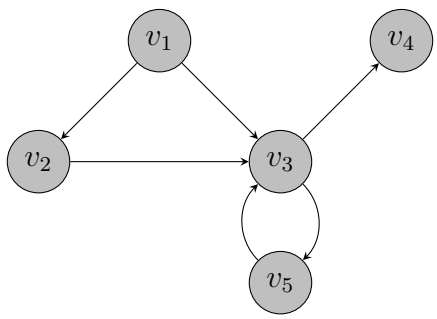
Definition 3.15 A graph $G = (V, E)$ with $V = \{1, 2, \ldots, n\}$ can be represented by its **adjacency matrix**

$$\mathbf{A} = [a_{i,j}]_{i,j=1}^{n} \in \mathbb{R}^{n \times n},$$

where

$$a_{i,j} = \begin{cases} 1, & \text{if } (i,j) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

Example 3.16 The directed graph



owns the adjacency matrix

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

The adjacency matrix of the underlying undirected graph is

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

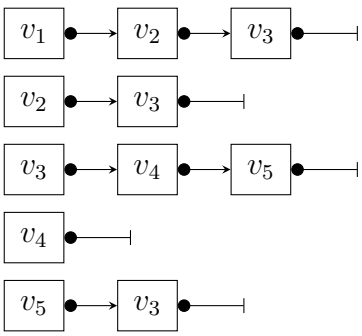
△

Remark In case of undirected graphs, the adjacency matrix $\mathbf{A}$ is always symmetric, meaning that $a_{i,j} = a_{j,i}$ for all $1 \leq i, j \leq n$.

The memory requirement of the adjacency matrix is $|V|^2$, independent of the number of edges $|E|$. A more memory efficient representation of the graph $G = (V, E)$ is obtained therefore by its *adjacency lists*:

Definition 3.17 The **adjacency list** of the vertex $v \in V$ contains the vertex $w \in V$ if and only if $(v, w) \in E$.

Example 3.18 The adjacency lists of the directed graph from Example 3.16 are given as follows:



△

Adjacency lists can be stored as (singly) *linked lists*, where each element has the form

```
struct node
{  unsigned int  number;
   struct node  *next;
   };
```

If `A` and `B` are of type `struct node`, then `A.number` and `B.number` contain the number of the vertex, while the assignment

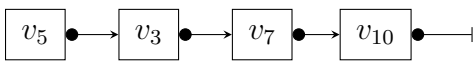
```
A.next = &B;
```

defines `B` as the successor of `A`. Instead of `B`, one can thus write `A.next`. The end of the list is indicated by

```
B.next = NULL;
```

Only the heads of the list need to be stored, i.e., the first elements. This can be done by an array of length $|V|$.

Remark Another possibility to store lists provides the representation through two arrays `number[n]`, `next[n]`, and the variable `head`. For example, the list



can also be realized by

	number	next
1	7	4
2	3	1
3	5	2
4	10	0

`Kopf: 3`

The adjacency lists have the memory requirement $|V| + |E|$ in case of directed graphs. As every edge appears twice, the memory requirement is $|V| + 2|E|$ in case of undirected graphs.

3.6 Graph traversal

Often a process of visiting, checking, and/or updating each vertex of a graph is needed. Popular graph traversal algorithms are *depth-first search* and *breadth-first search*. Both searches can be traced back to the following algorithm, which visits for a given vertex s all the reachable vertices $v \in \text{post}^\star(s)$.

Algorithm 3.19 (Graph traversal)
input: graph $G = (V, E)$ and initial vertex $s \in V$
output: acyclic graph $G' = (R, T)$ with $R = \{s\} \cup \text{post}^\star(s)$ and $T \subset E$

① Initialization: $R := \{s\}$, $Q := \{s\}$, $T = \emptyset$.
② If $Q = \emptyset$, then `stop`, otherwise choose $v \in Q$.
③ Choose $w \in V \setminus R$ with $e = (v, w) \in E$. If no such w exists, then set $Q := Q \setminus \{v\}$ and go to ②.
④ Set $R := R \cup \{w\}$, $Q := Q \cup \{w\}$, $T = T \cup \{e\}$, and go to ②.

Theorem 3.20 Algorithm 3.19 yields an acyclic graph $G' = (R, T)$ with $R = \{s\} \cup \text{post}^\star(s)$ and $T \subset E$.

Proof. The graph (R, T) is connected at each point of time of the algorithm. Especially, (R, T) is acyclic, since the edge $e = (v, w)$ connects always the vertices $v \in Q \subset R$ and $w \in V \setminus R$.
Assume that, when the algorithm terminates, there exists a vertex $w \in V \setminus R$ which is reachable from s. Let $\pi = s, v_1, \ldots, v_r, w$ be a simple s-w-path in G and $(x, y) \in E$ an edge with $x \in R$ and $y \notin R$. Since $x \in R$, there must hold $x \in Q$ at some point of time during the execution of the algorithm. The algorithm does however not terminate until x is removed from Q. This happens only if $(x, y) \notin E$. □

Theorem 3.21 Let the executions of "choose $v \in Q$" and "choose $w \in V \setminus R$ with $e = (v, w) \in E$" require $\mathcal{O}(1)$ operations. Then, Algorithm 3.19 has the complexity

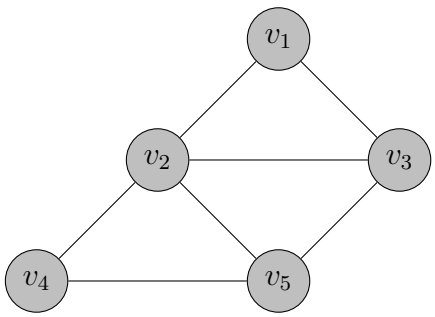
$\mathcal{O}(|V| + |E|)$.

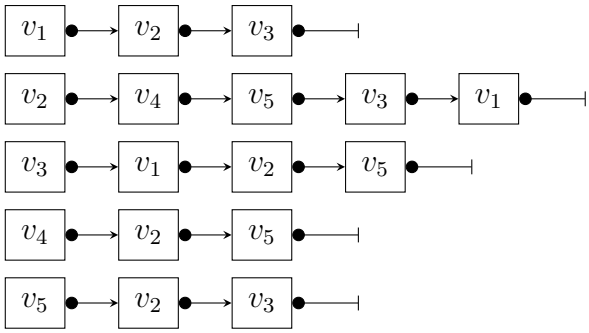
Proof. Each vertex $v \in V$ is visited at most $(|\operatorname{post}(v)| + 1)$-times and each each $e \in E$ at most one times. $\square$

Depending on how the set Q is organized and managed, different results are obtained. We distinguish:

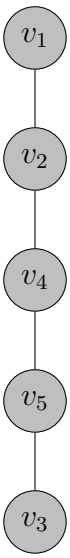
Definition 3.22 The **depth-first search** is obtained when that vertex $v \in Q$ is chosen which has been lastly added to Q. Whereas, the **breadth-first search** is obtained when that vertex $v \in Q$ is chosen which has been firstly added to Q.

Example 3.23 Let the graph $G = (V, E)$ be given by

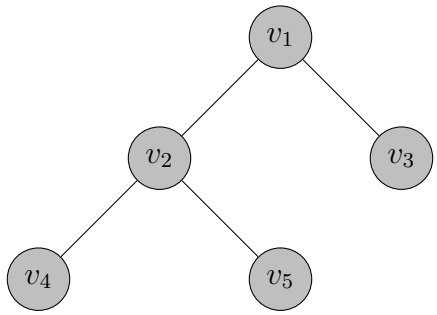




and $s = v_1$. The depth-first search yields the visit order v_1, v_2, v_4, v_5, v_3 and hence the tree



Whereas, the breadth-first search results in the visit order v_1, v_2, v_3, v_4, v_5, which corresponds to the tree



△

Theorem 3.24 Let $G = (V, E)$ be an undirected graph, $s, v \in V$, and

$$\mathrm{dist}_G(s, v) := \min\{|\pi| : \pi = s, u_1, \ldots, u_r, v \text{ path in } G\}.$$

Then, the BFS-graph $G' = (R, T)$ with respect to the starting vertex $s \in V$ contains a shortest path for each $v \in \mathrm{post}^\star(s)$. This means that the simple path $\pi = s, u_1, \ldots, u_r, v$ satisfies $|\pi| = \mathrm{dist}_G(s, v)$.

Proof. We first note that π is uniquely determined as $G' = (R, T)$ is acyclic. We modify Algorithm 3.19 as follows: We set $\ell(s) := 0$ in ① and $\ell(w) := \ell(v) + 1$ in ④. Then, there obviously holds at any point of time

$$\ell(v) = \mathrm{dist}_{(R,T)}(s, v) \quad \text{for all } v \in R.$$

Moreover, if a $v \in Q$ is chosen in ②, then there is no $w \in R$ such that

$$\ell(w) > \ell(v) + 1. \tag{3.2}$$

We assume that the algorithm terminates and there exists a vertex $w \in V$ with

$$\mathrm{dist}_G(s, w) < \mathrm{dist}_{G'}(s, w).$$

If there is more than one such vertex, then we choose the one with the smallest distance $\mathrm{dist}_G(s, w)$. Let $\pi = s, u_1, \ldots, u_r, v, w$ be a shortest path in G. Thus, there holds $\mathrm{dist}_G(s, v) = \mathrm{dist}_{G'}(s, v)$ and $(v, w) \in E$. Further, it is

$$\begin{aligned}\ell(w) &= \mathrm{dist}_{G'}(s, w) > \mathrm{dist}_G(s, w) = \mathrm{dist}_G(s, v) + 1 = \mathrm{dist}_{G'}(s, v) + 1 \\ &= \ell(v) + 1.\end{aligned}$$

It follows $w \notin R$ according to (3.2) at the point of time when $v \in Q$ is removed. This, however, contradicts ③ due to $(v, w) \in E$. □

Remark According to Theorem 3.21, the above algorithm computes the values $\mathrm{dist}_{G'}(s, v) = \mathrm{dist}_G(s, v)$ for all $v \in R$ in complexity $\mathcal{O}(|V| + |E|)$.